

ITERATIVE NEGAMAX IN CONNECT FOUR

PEDRO D. LLERENAS

ABSTRACT. We implement a bot for the game Connect Four in C++ using bitboards and a non-recursive negamax with alpha-beta pruning.

1. INTRODUCTION

In this article, we present an implementation of a move searching bot for the game Connect Four® by Hasbro in C++. We use bitboards for the representation of the 6×7 board, and perform tree searches using the Negamax algorithm with alpha-beta pruning. Moreover, we accomplish this non-recursively, which allows for better memory management.

This article does not cover optimal strategies. If one wishes to truly understand how play with perfect strategy, refer to [1]. It is also important to note that this game has been solved, meaning that every possible position has a determined outcome, assuming optimal play from both players. This was first done by James Dow Allen (as mentioned in [1]), and independently by Victor Allis in 1988. These solutions used knowledge databases, as computers back then were not powerful enough to brute force the solution. One key takeaways is that with perfect play, Player 1 can always win by placing the first chip in the center.

Once computational power was enough, John Tromp [2] used about 40,000 computational hours to solve any 8-ply position.

More improvements on the solving algorithm can be explored in [3].

2. PRELIMINARIES

In this section, we present (almost) every data structure and algorithm needed for the functioning of our program.

2.1. Connect Four.

The game is a 2-player zero-sum game with the following board:

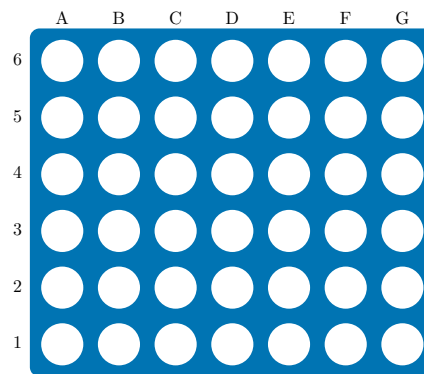


FIGURE 1. Empty Connect Four board.

This is a turn-based game, meaning one player performs a move, then the opponent makes a move. A move consists of picking one file (column) and *dropping* a chip. The files will be enumerated from A to F, from left to right. We will assume Player 1 uses red chips, while Player 2 uses yellow chips.

For example, if Player 1 starts the game by choosing file A, this is the resulting board:

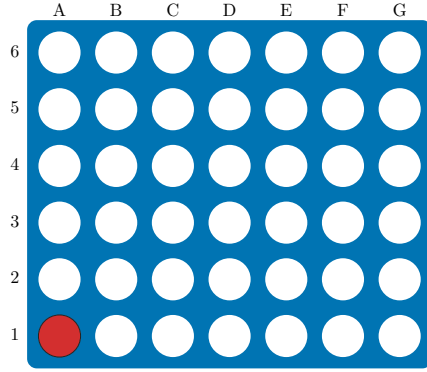


FIGURE 2. Player 1 plays File A.
(1. A1)

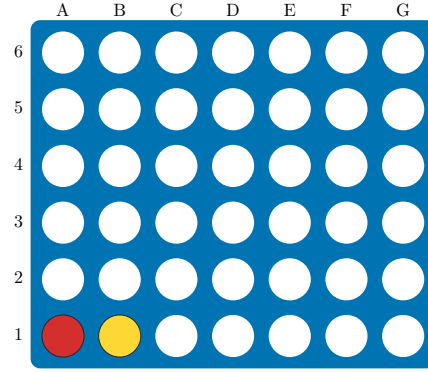


FIGURE 3. Player 2 plays File B.
(1. A1 b1)

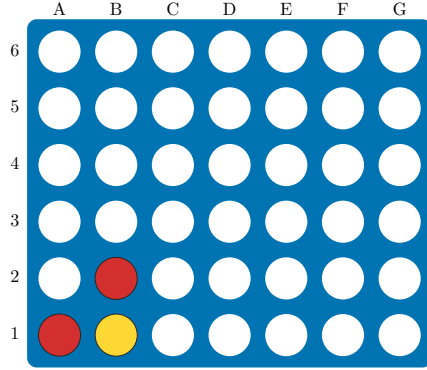


FIGURE 4. Player 1 plays File B.
(1. A1 b1 2. B2)

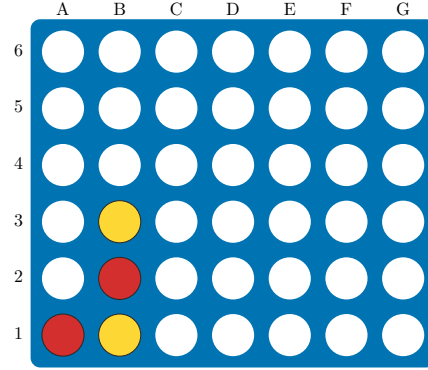


FIGURE 5. Player 2 plays File B.
(1. A1 b1 2. B2 b3)

The notation shown in the captions is similar to that of chess, where the first number (index) represents the move number, the letter represents the file and the adjacent number the rank. Capitalization is what distinguishes between the first player and the second player.

The goal of the game is to reach a position where there are 4 chips of the same color are connected diagonally, vertically, or horizontally. For example:

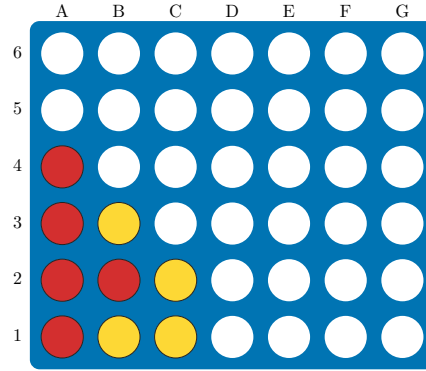


FIGURE 6. Board where Player 1 (red) wins by connecting 4 vertically in file A.

2.2. Bitboards.

Bitboards are board representations relying solely on bits [4]. More precisely, each of the $6 \times 7 = 42$ spaces in the Connect Four board are represented by a single bit (this attains the value of 0 or 1) of information. In C++, the data type `uint64_t` is an unsigned integer with 64 bits of capacity. Since the next smallest type is only 32 bits (`uint32_t`), we will have to conform ourselves with wasting some bits.

2.2.1. Why bitboards instead of arrays?

We *could* use arrays, but in an array, say we want to store both pieces, by marking each element as either 0, 1, 2, representing no chip, a Player 1 chip, and a Player 2 chip, respectively. Then, we would need at least 1 byte = 8 bits per board space. This is around 8 times more space than what is needed for a bitboard. Moreover, accessing a single bit from a `uint64_t` requires at most 2 machine instructions:

- 1 bit shift,
- 1 AND (&) operation.

On the other hand, accessing an array requires pointer arithmetic and loading the memory. If not cached, the accessing becomes orders of magnitude slower.

2.2.2. Board representation.

In our hardware, all 64 bits are contiguous. However, we use a certain rearrangement in order to make visualization easier. The following arrangement will be assumed henceforth.

5	12	19	26	33	40	47
4	11	18	25	32	39	46
3	10	17	24	31	38	45
2	9	16	23	30	37	44
1	8	15	22	29	36	43
0	7	14	21	28	35	42

LISTING 1. Bit alignment for board representation.

This means that only the first 49 bits of the 64 will be used. The dots at the top of the row are also part of the representation, and serve as *sentinel* values, that is, values whose purpose is to delimit the information in that file.

Since bitboards can only hold one value per bit, we cannot represent a board position with only one bitboard. Therefore, we will use two. One will be used to represent all occupied spaces (by both players), and the other one will only hold information about the player who is next to move. This is enough because it is possible to recover the opponents board from the following operation:

$$\text{opponent} = \text{current} \wedge \text{occupied}.$$

An alternative is to store Player 1 and Player 2 chips. However, spaces occupied by both will be revealed to be more useful later on, and thus required less total operations.

2.2.3. Example.

As an example, we will represent what Figure 6 would look like in terms of bitboards:

```

. . . . .
0 0 0 0 0 0 0
0 0 0 0 0 0 0
1 0 0 0 0 0 0
1 0 0 0 0 0 0
1 1 0 0 0 0 0
1 0 0 0 0 0 0

```

LISTING 3. Board representation for Player 1.

```

. . . . .
0 0 0 0 0 0 0
0 0 0 0 0 0 0
1 0 0 0 0 0 0
1 1 0 0 0 0 0
1 1 1 0 0 0 0
1 1 1 0 0 0 0

```

LISTING 4. Board representation of spaces occupied by both players.

Therefore, if we perform an XOR operation of these boards, we arrive to

```

. . . . .
0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 1 0 0 0 0 0
0 0 1 0 0 0 0
0 1 1 0 0 0 0

```

LISTING 5. XOR operation of previous boards. This is precisely the representation of Player 2's chips.

2.3. Data structures.

2.3.1. Trees.

The most important data structure in this article is an *m-ary tree*. Although it will not be explicitly implemented, it will be implicitly used as a reference for traversing possible future positions.

For our specific application, we will be working with 7-ary trees. This is because at each position, there are at most 7 possible moves. A 7-ary tree is a directed graph that holds information in each node; its children and data about the position it represents.

A tree can therefore represent all possible outcomes of a match. However, the amount of information in said tree will be in the order of 7^{42} . Therefore, it is realistically impossible (in terms of time) to traverse the whole tree in search of the best outcome at each possible position.

2.3.2. *Stack.*

A stack is an abstract data type that serves as a collection of elements with four operations:

- **Push:** adds an element to the collection,
- **Pop:** removes the most recently added element,
- **Top:** reads the most recently added element,
- **Size:** returns the size of the collection.

This data structure will be used to store a certain number of consecutive moves, which will serve as a backwards propagation mechanism to update the best possible move depending on the information found.

3. STORING A POSITION

Earlier we discussed the usage of bitboards to represent a position. Given that we will be storing multiple positions at a time, it is convenient to use classes to encapsulate each position. The usage of classes will also improve the readability of our code, as class methods allow us to operate over a particular position.

The Position class will be (roughly) defined as follows:

```
class Position {
private:
    Bitboard currentPieces;
    Bitboard bothPieces;
    uint8_t ply;
public:
    void play(File f);
    bool is_winning_move(File f) const;
    bool is_valid(File f) const;
};
```

LISTING 6. Snippet of the position class.

The `ply` property simply keeps track of the move number (counting half moves) of that board. Note that this is equivalent to the number of chips present in the board. The `play` method will update the bitboard representation of the position to reflect what playing `File f` would entail. The `is_winning_move` method will determine if playing in `File f` would result in a terminal position. The `is_valid` method will determine if playing in `File f` is possible. That is, if there enough space left in that file.

4. POSITION EVALUATION

Given that we will not be exploring the whole decision tree, we will have to define a *heuristic* function. That is, a *good enough* approximation to the true value of a position. It is a strategy used to reduce the search space of an algorithm, using

human knowledge about the game. In this case, we will artificially assign values to certain patterns without truly knowing the true outcome.

4.1. Terminal Positions.

A *terminal position* is one where a 4-connection has been made (see Figure 6). We will score this as 32000, and it will be the highest possible score attainable by a player.

4.2. Mate in 1.

A mate in 1 is classified as a position where the player can win in the next move. This will also be scored as 32000, as it is equivalent to a terminal state or it is known to be a terminal state before performing the move.

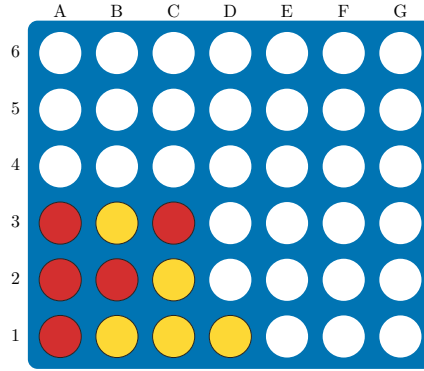


FIGURE 7. Red to play, can win in A4

4.3. Threats.

A *threat* is a pattern where only one same colored chip is missing, but an immediate win is not necessarily present. These have a weight of 1000.

4.3.1. Examples.

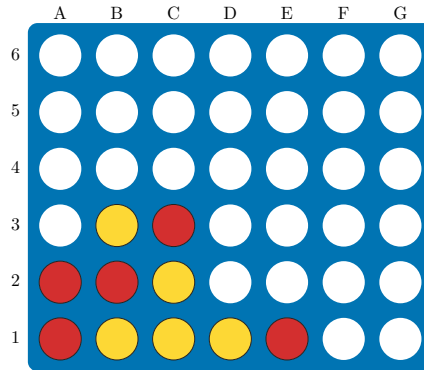


FIGURE 8. Red to play. Board with threats from both players, but red cannot win in the next move.

In the previous example, red has the threat of D4, while yellow has the threat of A4. Notice that neither A1 or E1 will count, as they are already blocked.

4.4. Center Control.

Similar to chess, having control over the center creates a big advantage. In our case, we may use the center file to create connections from both sides of the board. Therefore, we will reward the player that uses the files close to the center, as this heuristically allows for more winning positions. We score each chip in the center with 10 points.

5. MOVE GENERATION

The possible moves in each position is something almost trivial to us when watching the board. However, given that we are using bitboards, we must find a way to find the open spots in a given position using only bitwise operations.

This actually turns out to be easy. Using the bitboard mask of both occupancies, adding the following bitboard mask to it will result in the smallest rank available in each file:

.	.	.	.	.	.	.
0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0
1	1	1	1	1	1	1

LISTING 7. A bitboard mask of the bottom rank.

The reason why this works is because of carry-over's when performing the sum. This each file has 1's for each occupied position, adding a 1 will set every bit to 0 except the smallest index where a 0 is found.

An optimization can be done when generating movements. Namely, we only consider moves which do not lead to an immediately losing position. That is, if there is a threat from the opponent, we only consider the move which blocks the threat. If every move results in an immediate loss, we return no movements.

5.0.1. Examples.

If we have the following position:

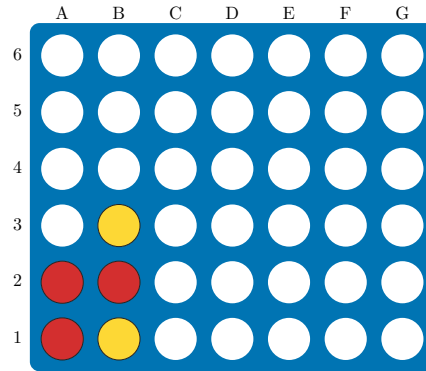


FIGURE 9.

Yellow is to move. Since they have no immediate wins, we search for red's immediate wins. Since they have none, the corresponding bitboard with moves to be considered is given by

.	.	.	.	.	.	.
0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	1	0	0	0	0	0
1	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	1	1	1	1	1

LISTING 8. Mask of the non-losing moves from position Figure 9.

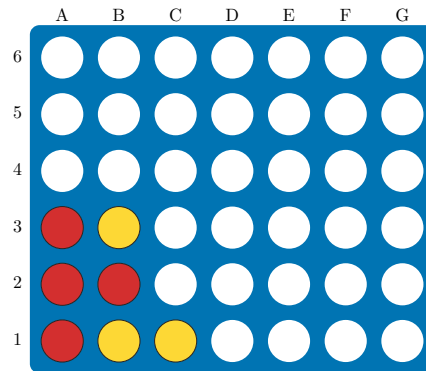


FIGURE 10. Example position for move generation.

Yellow has the next move. Since yellow has no immediate wins, his only move to consider will be A1. Every other move results in red winning in their turn. Therefore, the bitboard containing the moves to be considered is given by

.	.	.	.	.	.	.
0	0	0	0	0	0	0
0	0	0	0	0	0	0
1	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0

LISTING 9. Mask of the non-losing moves from position Figure 10.

6. SEARCH ALGORITHM

6.1. Minimax.

When one plays Connect Four, one seeks to maximize their probability of winning. This can be done by playing hypothetical moves and try to arrive to a conclusion that favors you the most. However, the opponent will most likely also do this, so we cannot assume that the opponent will play the worst move. That is, we try to maximize our own score, while the opponent tries to maximize theirs, this minimizing ours. This is known as the *minimax* algorithm.

As mentioned before, it is not feasible to do a whole search of the tree. Therefore, we will use a *heuristic* to evaluate a certain position, despite not knowing the exact outcome of said position, assuming perfect play from both players. Refer to Section 4 for a detailed explanation on how evaluations are done.

6.2. Negamax.

We start by noticing that, mathematically, $\min(a, b) = -\max(-a, -b)$. Therefore, minimizing the evaluations a, b is equivalent to the negative of the maximization of $-a, -b$. Thus, the *negamax* variant of minimax assumes that the score of the other player is the negative of their own. Therefore, it becomes redundant to differentiate between Player 1 and Player 2, so we may only focus on maximizing the negative scores of the current player's move.

6.2.1. Alpha-Beta pruning.

On its own, Negamax would search the whole position variation tree. However, this algorithm alone does not use the information from sibling nodes. That is, it searches the tree exhaustively, even if the best theoretical move —given a certain depth— has already been found in the first few iterations.

The alpha-beta pruning variation of the negamax search algorithm uses 2 extra variables at each node of the search tree. One describes the score of the best move the opponent can make, and the other contains the score of the best move you can make. These always start off at the worst possible values (∞ and $-\infty$). Once the maximum depth of the negamax algorithm has been reached, an evaluation is given, and a value is updated correspondingly. This value propagates upwards, and subsequent node searches will compare these newly set alpha-beta values to determine if searching that particular branch can yield a better score for the root node. If not, the branch is completely ignored, saving computational time.

According to [5], the time complexity of Negamax with alpha-beta pruning is $\mathcal{O}\left(7^{\frac{d}{2}}\right)$ when considering the best move first.

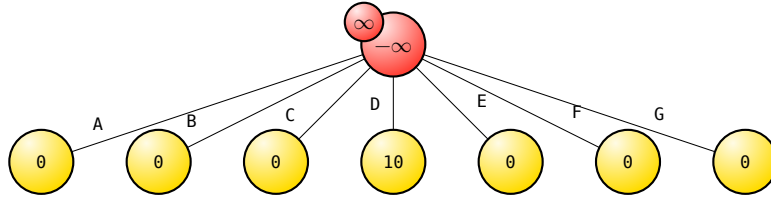


FIGURE 11. Negamax algorithm with depth 1 as a tree. The color indicates whose turn it is to move. The root node is the empty board. The big node denotes α , and the small node represents β . The root node is always initialized with $\beta = \infty$ and $\alpha = -\infty$. The yellow nodes are updated when the static evaluation is greater than α , which in this case any number is greater than $-\infty$.

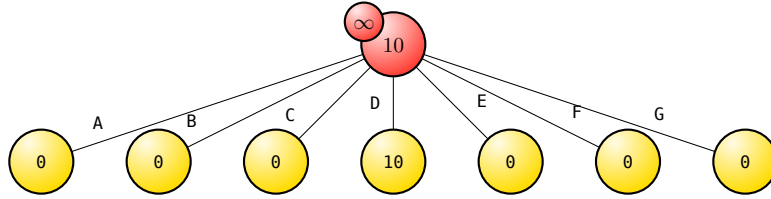


FIGURE 12. Root of the tree is updated with the best evaluation found. The best move is file D.

In this previous example, it can be seen that due to our evaluation function, the center is generally preferred. Therefore, if we reorder our moves in a way that the *heuristically* best nodes are searched first, we can update its parent node faster, and the information can be shared with the following explored nodes.

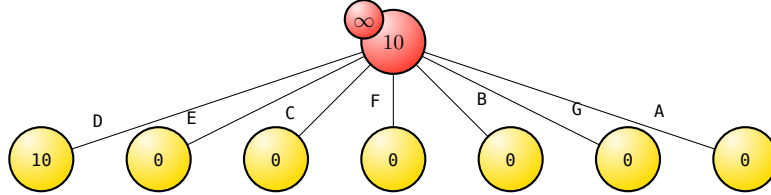


FIGURE 13. Moves ordered from *best* to *worst*. The value of α for the root node is immediately set to 10 after the first node visit, compared to it being updated only 4 nodes in for the first move arrangement.

There is no pruning done in the previous example because it is of depth 1. The following example is for demonstration purposes only; the values do not reflect a real game.

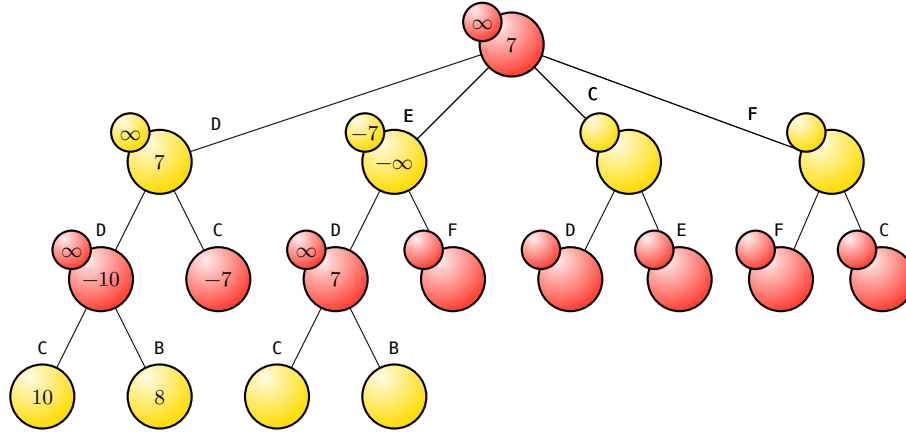
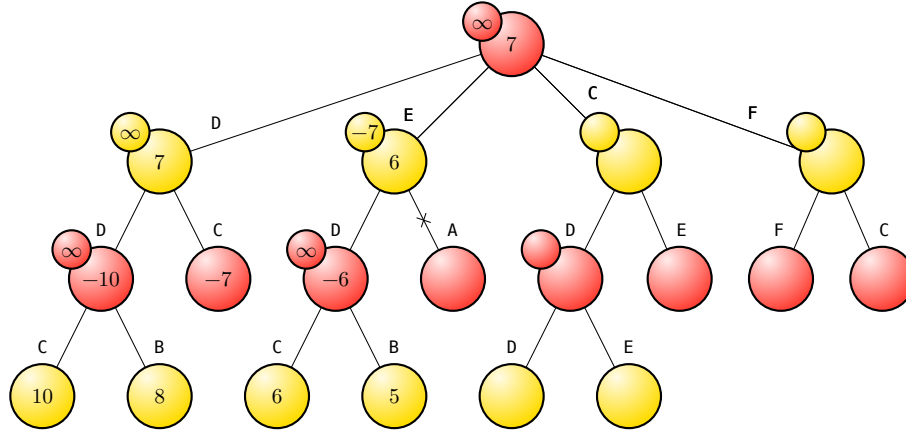


FIGURE 14. This tree shows the first few steps of the negamax process.

FIGURE 15. Continuation of the process of the previous tree. Here we have pruned the marked branch, due to the updated α being greater than registered β .

The previous tree continues being searched, but we stop our illustration there. Note that even though we have the whole tree mapped out, *a priori* we do not know where the branches will end.

6.3. Implementation.

Although the recursive solution to this problem is almost trivial by the construction of the problem, setting up an iterative solution brings some benefits to it. For example, one has complete control over the size of the stack, the size of the nodes, and parallelizing the process becomes much easier.

For the search tree, we will be using the following `struct` to keep track of the information at each node:

```

struct C4Node {
    Position pos;
    int16_t alpha, beta;
    uint8_t depth;
    uint8_t move_index;
    Bitboard moves;
    uint8_t move_count;
    int16_t best_score;
    Bitboard best_move;
};

```

LISTING 10. Data structure for search tree

Given this definition, we provide a pseudocode of the implementation of our iterative search.

ALGORITHM 1. Iterative Negamax with Alpha-Beta pruning

```

1: procedure NEGAMAX( $P, \alpha, \beta, D$ )
2:    $s \leftarrow \text{Stack}\langle \text{C4Node} \rangle$ 
3:    $s \leftarrow \text{push}(\text{C4Node}(P, \alpha, \beta, \dots))$ 
4:
5:   while ! $s.\text{empty}()$  do
6:      $\text{curr} \leftarrow s.\text{top}()$ 
7:     if  $\text{is\_terminal}(\text{curr}) \parallel \text{curr.depth} == D$  then
8:       if  $\text{no\_moves}$  then
9:          $\triangleright$  If no moves are available, then this position is lost
10:         $\text{score} \leftarrow -32000$ 
11:       else if  $\text{depth} == 42$  then
12:          $\triangleright$  If position is full
13:          $\text{score} \leftarrow 0$ 
14:       else if  $\text{curr.depth} == \text{max\_depth}$  then
15:          $\text{score} \leftarrow \text{heuristic}(P)$ 
16:       end
17:        $\text{root\_move} \leftarrow \text{curr.best\_move}$ 
18:        $s \leftarrow \text{pop}()$ 
19:       if  $s.\text{empty}()$  then
20:         return  $\text{root\_move}$ 
21:       end
22:        $\text{C4Node parent} \leftarrow s.\text{top}()$ 
23:        $\text{score} \leftarrow -\text{score}$ 
24:        $\triangleright$  Beta cutoff
25:       if  $\text{parent.alpha} \geq \text{parent.beta}$  then
26:         if  $s.\text{empty}()$  then
27:           return  $\text{parent.best\_move}$ 
28:         end
29:          $s.\text{top}().\text{move\_index} \leftarrow 7 \triangleright$  forces search of this tree branch to end
30:       end
31:     end
32:      $\text{moves} \leftarrow \text{Array}\langle \text{Bitboard} \rangle$   $\triangleright$  Ordered moves
33:      $s \leftarrow \text{push}(\text{child})$   $\triangleright$  Push position with the next move
34:     return null
35: end

```

7. APPLICATION USAGE

7.1. Compilation.

We have separated the application in 7 header files and 6 source files. To compile, one can either use the makefile contained in the source folder, or manually include all source files. For the former, simply use

```
make c4 && ./c4
```

For the latter compilation, use

```
g++ main.cpp bitboard.cpp position.cpp evaluate.cpp search.cpp movegen.cpp
-o c4 && ./c4
```

After executing the binary, the user will be greeted with the following command line interface:

```
+---+---+---+---+---+---+
|   |   |   |   |   |   | 6
+---+---+---+---+---+---+
|   |   |   |   |   |   | 5
+---+---+---+---+---+---+
|   |   |   |   |   |   | 4
+---+---+---+---+---+---+
|   |   |   |   |   |   | 3
+---+---+---+---+---+---+
|   |   |   |   |   |   | 2
+---+---+---+---+---+---+
|   |   |   |   |   |   | 1
+---+---+---+---+---+---+
  a   b   c   d   e   f   g
```

Enter a move (column A-G).

Once the user enters a column to play, the bot will take some time to find the best move, and a new position is printed. For example, if we enter D, the following board will appear.

```
+---+---+---+---+---+---+
|   |   |   |   |   |   | 6
+---+---+---+---+---+---+
|   |   |   |   |   |   | 5
+---+---+---+---+---+---+
|   |   |   |   |   |   | 4
+---+---+---+---+---+---+
|   |   |   |   |   |   | 3
+---+---+---+---+---+---+
|   |   |   | X |   |   | 2
+---+---+---+---+---+---+
|   |   |   | 0 |   |   | 1
+---+---+---+---+---+---+
  a   b   c   d   e   f   g
```

Enter a move (column A-G).

After many moves, one can arrive to a position such as

+---+---+---+---+---+---+---+						
			X		X	
+---+---+---+---+---+---+---+						
		0		X		0
+---+---+---+---+---+---+---+						
		X		X		X
+---+---+---+---+---+---+---+						
	X		X		0	
+---+---+---+---+---+---+---+						
	0		X		X	
+---+---+---+---+---+---+---+						
	X		0		0	
+---+---+---+---+---+---+---+						
	0		X		0	
+---+---+---+---+---+---+---+						
	X		0		0	
+---+---+---+---+---+---+---+						
	a	b	c	d	e	f

You lose!

With optimal strategy (which I do not possess), one can most likely beat the heuristic function, as we do move first.

8. POSSIBLE IMPROVEMENTS

8.1. Transposition tables.

Transposition tables are hash tables that store previously explored positions together with their evaluation. One flaw (or inefficiency) of the negamax algorithm is that once a move is played, it loses memory of what it had already calculated. This is problematic, as we are wasting computational time on positions we already had the solution to. Thus, additional storage can be used to prevent calculations. To store them, one needs to hash the positions. A map suggested by [3] is doing position + mask + bottom, where position are the bits for one of the players, mask is bits for both players, and bottom is a mask of the first row.

8.2. Iterative deepening.

This technique consists in exploring at a shallow depth, then iteratively exploring deeper. This allows possible shallow victories, but also the pruning process becomes more efficient.

Further improvements can be found in [3].

REFERENCES

1. Allen, J.D.: The Complete Book of Connect Four, (1990)
2. Tromp, J.: John's Connect Four Playground, (1995)
3. Pons, P.: Solving Connect 4: How to build a perfect AI, (2019)
4. Contributors, C.W.: Bitboards
5. Contributors, C.W.: Bitboards

MAESTRIA EN CIENCIAS DE LA COMPUTACION, CENTRO DE INVESTIGACION EN MATEMATICAS
Email address: pedro.llerenas@cimat.mx
URL: <https://github.com/pdllerenas/ConnectFour>